

Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU

Assignment 1: Code Execution

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo
Student HPC-Account: ptfs356v

Gehan Younes
Student IdM: qi39dyry
Student HPC-Account: ptfs437v

May 7, 2026

1. Pipelines (25 Credits)

We are given the following properties of a CPU pipeline:

- A CPU core has Two floating point `ADD` pipelines.
- The floating-point `ADD` pipeline has a **latency** of 4 cycles.
- The pipeline has a **throughput** of 1 result per cycle (i.e., when fully utilized).
- We are required to compute how many independent `ADD` instructions are needed to reach **90% of the maximum throughput**.

Pipeline Throughput Formula

The throughput of a pipelined functional unit is given by:

$$\text{Throughput} = \frac{N}{N + m - 1} \tag{1}$$

Where:

- N = Number of independent instructions
- m = Pipeline latency (in cycles)

We are given:

$$m = 4 \qquad \text{and} \qquad \text{Throughput} = 0.9 \text{ inst./cy}$$

Substituting into the equation:

$$0.9 = \frac{N}{N + 3} \tag{2}$$

Solving for N:

$$\begin{aligned} 0.9(N + 3) &= N \\ 0.9N + 2.7 &= N \\ 2.7 &= 0.1N \\ N &= 27 \end{aligned}$$

Answer

To achieve 90% of the maximum throughput on a CPU core with two 4-stage floating-point `ADD` pipelines, at least 27 independent `ADD` instructions are required *per pipeline*, giving a total of 54 independent `ADD` instructions overall.

This result matches the pipeline utilization model from the lecture, where performance improves as the number of independent instructions increases relative to the pipeline depth.

2. More Pipelines (35 Credits)

We are given three loops, each updating array elements based on the value two positions prior:

```
double a[...], b[...], c[...];
double s = 0.1;
for (int i = 2; i < N; ++i) { a[i] += s * a[i-2]; }
for (int i = 2; i < N; ++i) { b[i] += s * b[i-2]; }
for (int i = 2; i < N; ++i) { c[i] += s * c[i-2]; }
```

Listing 1: Three separate loops with stride-2 recurrence

(a) Single Loop Performance

Instruction Breakdown

Each iteration of a single loop (`a[i] += s * a[i-2]`) consists of:

- `LOAD a[i-2]`
- `MULT s * a[i-2]` (latency: $m_{\text{MULT}} = 8$ cy)
- `LOAD a[i]`
- `ADD a[i] + temp` (latency: $m_{\text{ADD}} = 6$ cy)
- `STORE a[i]`

This yields **5 instructions** and **2 FLOPs** per iteration.

Throughput Bottleneck Analysis

Using the pipeline throughput formula:

$$\text{Throughput} = \frac{N}{N + m - 1} \tag{3}$$

where N is the number of independent instructions and m is the pipeline latency. For large N, this converges to 1 result/cy per pipeline. The per-cycle issue limits are:

- **LOAD**: 2 loads required, 1 port available ⇒ 2 cy/iter
- **MULT**: 1 multiply, 1 port available ⇒ 1 cy/iter
- **ADD**: 1 add, 1 port available ⇒ 1 cy/iter
- **STORE**: 1 store, 1 port available ⇒ 1 cy/iter
- **Instruction retirement**: 5 instructions at 4/cy ⇒ 1.25 cy/iter

The **LOAD unit** is the throughput bottleneck at **2 cy/iter**.

Recurrence (Latency) Analysis

The stride-2 access `a[i-2]` creates two independent recurrence chains (even and odd indices). The critical path latency through one chain step is:

$$L_{\text{chain}} = m_{\text{MULT}} + m_{\text{ADD}} = 8 + 6 = 14 \text{ cy} \tag{4}$$

Applying the throughput formula to the 2 independent chains, the recurrence limit per iteration is:

$$T_{\text{recurrence}} = \frac{L_{\text{chain}}}{N_{\text{chains}}} = \frac{14}{2} = 7 \text{ cy/iter} \tag{5}$$

Binding Bottleneck

$$T_{\text{iter}} = \max\left(T_{\text{LOAD}},\ T_{\text{recurrence}}\right) = \max(2,\ 7) = 7 \text{ cy/iter} \tag{6}$$

The loop is **recurrence-bound**. The expected performance is therefore:

$$\boxed{P_a = \frac{2 \text{ FLOPs}}{7 \text{ cy}} \approx 0.286 \text{ FLOP/cy}} \tag{7}$$

(b) Optimization: Loop Fusion

Transformation

The compiler can apply **loop fusion**, merging all three loops into one:

```
for (int i = 2; i < N; ++i) {
    a[i] += s * a[i-2];
    b[i] += s * b[i-2];
    c[i] += s * c[i-2];
}
```

Listing 2: Fused loop exposing more independent work

Recurrence Analysis After Fusion

Each array contributes 2 independent chains, giving $N_{\text{chains}} = 6$ total. Applying the throughput formula:

$$T_{\text{recurrence}} = \frac{L_{\text{chain}}}{N_{\text{chains}}} = \frac{14}{6} \approx 2.33 \text{ cy/iter} \tag{8}$$

The recurrence bottleneck is now effectively resolved.

Throughput Bottleneck Analysis (Fused)

The fused iteration contains $3 \times 5 = 15$ instructions and $3 \times 2 = 6$ FLOPs. Applying the throughput formula to each resource:

- **LOAD**: 6 loads, 1 port available ⇒ 6 cy/iter
- **MULT**: 3 multiplies, 1 port available ⇒ 3 cy/iter
- **ADD**: 3 adds, 1 port available ⇒ 3 cy/iter
- **STORE**: 3 stores, 1 port available ⇒ 3 cy/iter
- **Instruction retirement**: 15 instructions at 4/cy ⇒ 3.75 cy/iter

The **LOAD unit** remains the bottleneck at **6 cy/iter**.

Binding Bottleneck

$$T_{\text{iter}} = \max\left(T_{\text{LOAD}},\ T_{\text{recurrence}}\right) = \max(6,\ 2.33) = 6 \text{ cy/iter} \tag{9}$$

The fused loop is **LOAD-throughput bound**. The optimal performance is:

$$\boxed{P_b = \frac{6 \text{ FLOPs}}{6 \text{ cy}} = 1.0 \text{ FLOP/cy}} \tag{10}$$

Summary

Loop fusion improves performance by a factor of:

$$\text{Speedup} = \frac{P_b}{P_a} = \frac{1.0}{0.286} \approx 3.5\times \tag{11}$$

This reaches the hard ceiling imposed by the single LOAD port. No further scalar optimization can exceed 1 FLOP/cy without additional memory bandwidth or FMA support.

3. Square Root in Depth (40 Credits)

The integration loop from `pi_bench.c` approximates π using:

$$\pi = \int_0^1 4\sqrt{1-x^2}\,dx$$

implemented as:

```
for (int i = 0; i < N; i++) {
    x   = (i + 0.5) * delta_x;
    sum = sum + 4.0 * sqrt(1.0 - x * x);
}
```

(a) Required Compiler Optimization

The accumulation into the single variable `sum` creates a loop-carried data dependency: each iteration must wait for the previous `ADD` (or `FMA`) to complete before it can begin its own accumulation. This serializes the loop and prevents the CPU from keeping the SQRT pipeline busy.

The compiler must apply **Modulo Variable Expansion (MVE)**: the loop is unrolled by a factor k and the summation is split into k independent partial sums:

```
// k-way MVE
sum1 += 4.0 * sqrt(1.0 - x1 * x1);
sum2 += 4.0 * sqrt(1.0 - x2 * x2);
// ...
sumk += 4.0 * sqrt(1.0 - xk * xk);
// Final reduction
sum = sum1 + sum2 + ... + sumk;
```

Each partial accumulator carries an independent data-flow chain, allowing the out-of-order core to issue multiple `SQRTSD` instructions simultaneously and keep the SQRT pipeline busy. The unroll factor k must be large enough relative to the SQRT pipeline depth m so that the pipeline throughput formula:

$$\text{Throughput} = \frac{N}{N+m-1} \tag{12}$$

approaches its maximum of 1 result/cy. Since `SQRTSD` has a very long latency compared to standard FP instructions, a large number of independent instructions is required to saturate the pipeline.

Note that the compiler can only apply MVE if it is allowed to reorder floating-point operations, since FP addition is not associative. At `-O1` this transformation is typically **not** applied, which is why the single-accumulator bottleneck dominates at low optimization levels.

(b) IPC Calculation at $N = 10^9$

Machine Model

The measurements were performed on **Fritz** (Intel Xeon 8360Y, Ice Lake SP), whose scalar execution model is (lecture slide):

- 2 FMA/cy, 2 MULT/cy, 2 ADD/cy
- 2 LD/cy, 1 ST/cy
- **Maximum 5 instructions/cycle** $\Rightarrow$ CPI ≥ 0.20

Key instruction costs from the lecture slides:

Instruction	**Latency [cy]**	**Throughput [instr/cy]**
ADD DP	4	2
MULT DP	4	2
FMA DP	4	2
SQRT DP	26	$1/12 \approx 0.083$

Assumed Instruction Composition

With `-O1 -no-vec -xHost` (scalar, FMA enabled via `-xHost`), the compiler folds `sum += 4.0 * sqrt(...)` into one FMA, giving:

Instruction	**Purpose**	**Count**
`CVTSI2SD`	Convert loop counter i to double	1
`ADDSD`	$i + 0.5$	1
`MULSD`	$\times\, \delta_x \Rightarrow x$	1
`MULSD`	$x \times x$	1
`SUBSD`	$1 - x^2$	1
`SQRTSD`	$\sqrt{1 - x^2}$	1
`VFMADDSD`	`sum` $+= 4.0 \times \sqrt{\cdots}$	1
`INC`	$i \mathrel{+}= 1$	1
`CMP`	Compare i with N	1
`JL`	Conditional branch	1
Total		**10**

Theoretical Bottleneck Analysis

Following the lecture's $P_{\max}$ methodology, we identify the potential bottlenecks:

Recurrence bottleneck. The only loop-carried dependency runs through `sum`: the FMA result of iteration i is consumed by the FMA of iteration $i + 1$, giving:

$$T_{\text{recurrence}} = L_{\text{FMA}} = 4 \text{ cy/iter} \tag{13}$$

Note that `SQRTSD` is *not* part of the recurrence — each SQRT depends only on the loop counter i, so the out-of-order engine can overlap SQRT computations from multiple iterations.

Instruction throughput bottleneck. With 10 instructions per iteration and a maximum of 5 instructions/cy:

$$T_{\text{instr}} = \frac{10}{5} = 2 \text{ cy/iter} \tag{14}$$

Theoretical minimum.

$$T_{\text{iter}}^{\text{theory}} = \max(T_{\text{recurrence}},\ T_{\text{instr}}) = \max(4,\ 2) = 4 \text{ cy/iter} \tag{15}$$

This simplified theoretical minimum ignores a standalone SQRT throughput bound because the SQRT operations are not part of the loop-carried `sum` recurrence and can be overlapped across iterations by the out-of-order engine.

Measurement

The code was compiled and run on Fritz (base clock $f = 2.4 \times 10^9$ Hz):

```
icx -O1 -no-vec -xHost -lm -o pi_bench_2 pi_bench_2.c
./pi_bench_2
```

```
N         = 1000000000
Result    = 3.1415926536
Runtime   = 1.942359 seconds
cy/iter   = 4.66
```

IPC Calculation

Using the measured cycles per iteration and the assumed instruction count:

$$\text{IPC} = \frac{\text{Instructions per iteration}}{T_{\text{iter}}} = \frac{10}{4.66} \approx \boxed{2.15} \tag{16}$$

Discussion

The measured 4.66 cy/iter is close to the theoretical recurrence bound of 4 cy/iter, suggesting the loop is largely **recurrence-bound** through the `sum` accumulator. The small overhead above 4 cy/iter is consistent with the cost of the remaining instructions (the SQRT, MULTs, and loop mechanics) not being fully hidden by the out-of-order engine within the recurrence constraint. This confirms the need for MVE described in part (a): splitting `sum` into multiple independent accumulators would break the 4 cy recurrence bottleneck and allow the core to exploit more instruction-level parallelism.